

Cours d'introduction à la Data Science

Chargé du cours : Jules DEGILA, PhD, MCF
Assistante : Aurélie KPOZE, PhD

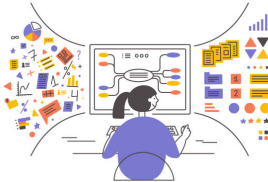
jules.degila@imsp-uac.org, satou.kpoze@imsp-uac.org

23 Janvier 2026



Chapitre 5

Fondamentaux du Deep Learning



Concepts clés, architectures et méthodes d'entraînement des réseaux de neurones profonds.

Plan du Chapitre 5

- ➊ Rappels sur le Machine Learning et limites des modèles classiques
- ➋ Neurone artificiel et perceptron
- ➌ Réseaux de neurones multicouches (MLP)
- ➍ Fonctions d'activation
- ➎ Fonction de coût et optimisation
- ➏ Rétropropagation du gradient (Backpropagation)
- ➐ Entraînement d'un réseau de neurones
- ➑ Problèmes courants : surapprentissage, underfitting, régularisation
- ➒ Introduction aux architectures profondes (CNN, RNN)

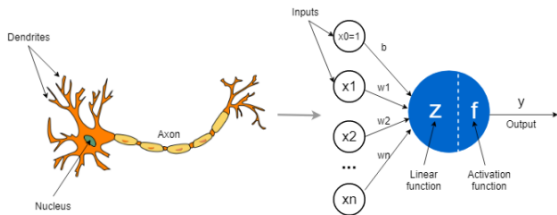
À la fin du chapitre, vous serez capables de :

- Comprendre les bases du Deep Learning
- Modéliser un neurone artificiel
- Expliquer l'apprentissage par rétropropagation
- Identifier les problèmes et limites des réseaux de neurones
- Découvrir les architectures profondes modernes



Introduction

- Les premiers travaux sur les neurones artificiels remontent à 1943 avec le modèle de McCulloch et Pitts.
- En 1958, Rosenblatt propose le perceptron
- Dans les années 1980, l'algorithme de rétropropagation permet enfin d'entraîner des réseaux de neurones profonds.



- Faute de données et de puissance de calcul, ces méthodes sont peu utilisées dans les années 1990.
- À partir de 2006, l'augmentation des données et de la puissance de calcul relance le Deep Learning.
- Pour des problématiques centrales en IA comme la reconnaissance vocale ou d'images; l'apprentissage automatique est inefficace
- Depuis, le Deep Learning s'est imposé comme une technologie centrale de l'intelligence artificielle moderne.

Introduction

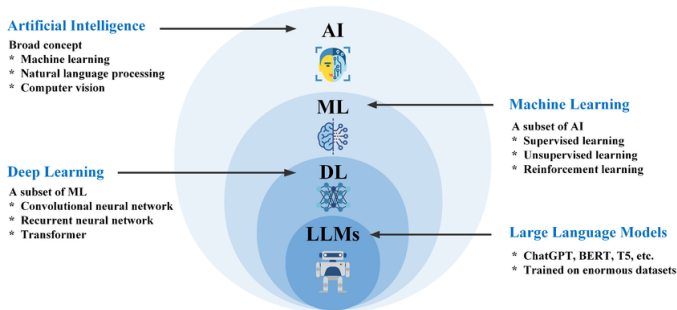


Figure: *

Image source : Yang, Zheng et al. (2025). *AI-Driven Safety and Security for UAVs*.

- **Définition générale** : Le Deep Learning (apprentissage profond) est un sous-domaine du Machine Learning qui utilise des réseaux de neurones artificiels composés de plusieurs couches pour apprendre automatiquement des représentations à partir des données.
- **Exemple** : Un système de reconnaissance d'images capable d'identifier des visages ou des objets apprend directement à partir de milliers d'images sans que l'on définisse manuellement les caractéristiques.
- **Définition mathématique** : Un modèle de Deep Learning est une fonction paramétrée $f(x; \theta)$ composée de plusieurs transformations successives, où les paramètres θ sont appris en minimisant une fonction de coût :

$$\theta^* = \arg \min_{\theta} \mathcal{L}(f(x; \theta), y)$$

Rappels sur le Machine Learning et limites des modèles classiques

- Le Machine Learning apprend des modèles à partir des données.
- Les modèles classiques nécessitent une extraction manuelle des caractéristiques.
- Ils sont limités pour les données complexes et de grande dimension (Images, son, texte).
- Le Deep Learning apprend automatiquement les représentations utiles.

Machine Learning classique vs Deep Learning

Critère	ML classique	Deep Learning
Extraction des caractéristiques	Manuelle (faite par l'expert)	Automatique (apprise par le réseau)
Complexité des modèles	Faible à moyenne	Très élevée (modèles profonds)
Quantité de données requise	Faible à moyenne	Très grande
Puissance de calcul	Faible à modérée	Élevée (GPU souvent nécessaire)
Performance sur données complexes	Limitée	Très élevée
Interprétabilité	Souvent bonne	Plus difficile à interpréter
Exemples de modèles	Régression, SVM, k-NN, arbres	CNN, RNN, Transformers

Partie 1: Rappels mathématiques

Rappels d'algèbre linéaire : Vecteurs et matrices

- Un **vecteur** est noté $x \in \mathbb{R}^n$:

$$x = \begin{bmatrix} x_1 \\ x_2 \\ \vdots \\ x_n \end{bmatrix}$$

- Une **matrice** est notée $W \in \mathbb{R}^{m \times n}$:

$$W = \begin{bmatrix} w_{11} & \cdots & w_{1n} \\ \vdots & \ddots & \vdots \\ w_{m1} & \cdots & w_{mn} \end{bmatrix}$$

- En Deep Learning :
 - les entrées sont des vecteurs
 - les poids sont des matrices

- En Deep Learning, une couche calcule une **combinaison linéaire** des entrées :

$$z = Wx + b$$

- Où :
 - $x \in \mathbb{R}^n$ est le vecteur d'entrée
 - $W \in \mathbb{R}^{m \times n}$ est la matrice de poids
 - $b \in \mathbb{R}^m$ est le vecteur de biais
- Le produit matriciel est défini par :

$$z_i = \sum_{j=1}^n W_{ij}x_j$$

- Cette opération est au cœur de tous les réseaux de neurones.

Rappels d'algèbre linéaire : Tenseurs

- Un **tenseur** est une généralisation :
 - scalaire : ordre 0
 - vecteur : ordre 1
 - matrice : ordre 2
 - tenseur : ordre ≥ 3
- Exemples :
 - Scalaire : $x \in \mathbb{R}$
 - Vecteur : $x \in \mathbb{R}^n$
 - Matrice : $W \in \mathbb{R}^{m \times n}$
 - Tenseur : $T \in \mathbb{R}^{d_1 \times d_2 \times d_3 \times \dots}$
- Un tenseur est donc un tableau de nombres à plusieurs dimensions.
- On parle aussi de :
 - **forme (shape)** du tenseur
 - **dimension** ou **ordre** du tenseur

- En Deep Learning, toutes les données sont représentées sous forme de tenseurs.
- Exemples :
 - Une image en niveaux de gris : tenseur de forme (H, W)
 - Une image couleur : tenseur de forme $(H, W, 3)$
 - Un lot (batch) d'images : tenseur de forme $(B, H, W, 3)$
 - Une séquence de mots : tenseur de forme (T, D)
- Les poids des réseaux et les activations sont aussi des tenseurs.
- Les opérations dans les réseaux (convolution, attention, etc.) sont des opérations sur des tenseurs.

Tenseurs et notion de shape

- Un **tenseur** est une généralisation :
 - scalaire (0D), vecteur (1D), matrice (2D), tenseur (ND)
- Un tenseur est défini par sa **forme (shape)** :

$$(d_1, d_2, \dots, d_k)$$

- Exemples :
 - Image couleur : $(H, W, 3)$
 - Batch d'images : $(B, H, W, 3)$
 - Séquence de mots : (T, D)
- En Deep Learning, données, poids et activations sont des tenseurs.

- Une **variable aléatoire** X représente un phénomène incertain.
- Une **probabilité** $P(X = x)$ vérifie :

$$0 \leq P(X = x) \leq 1, \quad \sum_x P(X = x) = 1$$

- **Probabilité conditionnelle :**

$$P(A \mid B) = \frac{P(A \cap B)}{P(B)}$$

- En Machine Learning, on cherche souvent à estimer :

$$P(y \mid x)$$

- En classification, la sortie du réseau est une distribution (Softmax).

- La **dérivée** mesure la variation d'une fonction :

$$\frac{df(x)}{dx}$$

- Pour une fonction de plusieurs variables $J(\theta_1, \dots, \theta_n)$, on définit le **gradient** :

$$\nabla_{\theta} J = \left(\frac{\partial J}{\partial \theta_1}, \dots, \frac{\partial J}{\partial \theta_n} \right)$$

- Le gradient indique la direction de la plus forte augmentation de la fonction.
- En Deep Learning, on utilise le **gradient pour minimiser la fonction de coût**.

Partie 2: Concepts fondamentaux des réseaux de neurones

Neurone artificiel : intuition

- Le neurone artificiel est inspiré du neurone biologique.
- Il reçoit plusieurs entrées et produit une sortie.
- Chaque entrée est pondérée par un poids qui représente son importance.
- Le neurone calcule une somme pondérée, puis applique une fonction d'activation.
- Intuition : il combine des informations et prend une décision simple
- Un réseau de neurones est simplement un grand ensemble de ces neurones connectés.

Modèle mathématique du neurone

- Un neurone calcule d'abord une combinaison linéaire :

$$z = w_1x_1 + w_2x_2 + \cdots + w_nx_n + b = \sum_{i=1}^n w_ix_i + b$$

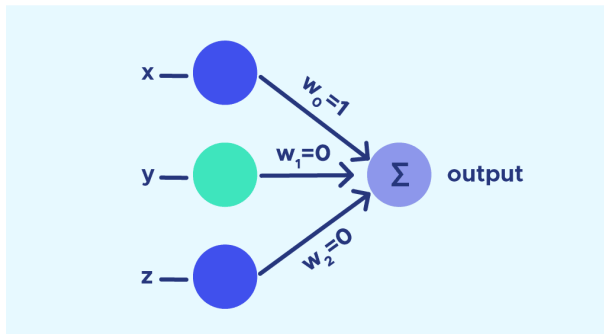
- Puis applique une fonction d'activation :

$$y = \varphi(z) = \varphi(w^T x + b)$$

- Où :
 - x : vecteur d'entrée
 - w : vecteur de poids
 - b : biais
 - φ : fonction d'activation

- Le perceptron est le plus ancien modèle de neurone artificiel.
- Il utilise une fonction d'activation seuil pour faire de la classification binaire.
- Il permet de séparer des données **linéairement séparables**.
- Cependant, il ne peut pas résoudre des problèmes non linéaires.
- Un exemple célèbre est le problème XOR.
 - Les données XOR ne sont pas séparables par une droite.
 - Donc un perceptron simple ne peut pas apprendre XOR.
- Cette limitation a motivé l'introduction des réseaux multicouches.

Perceptron

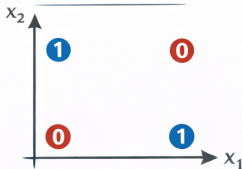


UN PERCEPTRON SIMPLE NE PEUT PAS APPRENDRE LE PROBLÈME XOR

DONNÉES XOR

x_1	x_2	XOR
0	0	0
0	1	1
1	0	1
1	1	0

PROBLÈME XOR



Aucune droite ne peut séparer les points correctement...

Points "0"

0 0

Points "1"

1 1



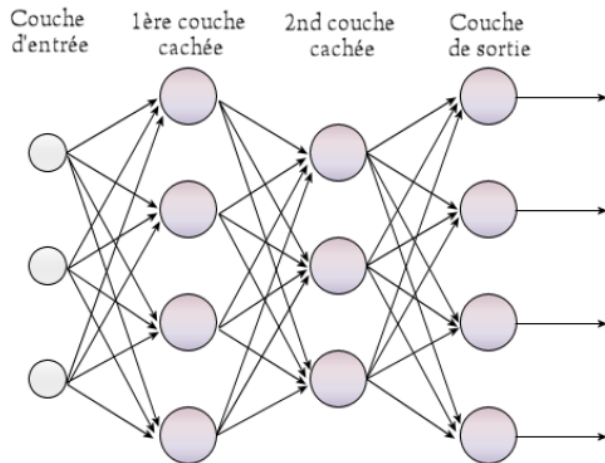
Perceptron Simple

Impossible pour XOR!

Réseaux de neurones multicouches (MLP)

- Un réseau de neurones multicouches (MLP : Multi-Layer Perceptron) est une extension du perceptron simple.
- Il est composé d'une couche d'entrée, d'une ou plusieurs couches cachées, et d'une couche de sortie.
- Chaque neurone d'une couche est connecté à tous les neurones de la couche suivante.
- Chaque couche applique une transformation linéaire suivie d'une fonction d'activation non linéaire.
- Les MLP permettent de modéliser des relations non linéaires complexes.
- Ils sont entraînés à l'aide de l'algorithme de rétropropagation du gradient.

Réseaux de neurones multicouches (MLP)



Modèle mathématique d'un MLP

- Un réseau multicouche applique une succession de transformations :

$$h^{(1)} = \varphi \left(W^{(1)}x + b^{(1)} \right)$$

$$h^{(2)} = \varphi \left(W^{(2)}h^{(1)} + b^{(2)} \right)$$

$\vdots$

$$y = f(x) = \varphi \left(W^{(L)}h^{(L-1)} + b^{(L)} \right)$$

- De manière compacte :

$$f(x) = \varphi_L \circ \varphi_{L-1} \circ \cdots \circ \varphi_1(x)$$

- Chaque couche effectue : transformation linéaire + non-linéarité.

Pourquoi utiliser plusieurs couches ?

- Une seule couche ne peut représenter que des frontières de décision linéaires.
- L'empilement de couches permet de construire des représentations hiérarchiques.
- Les premières couches apprennent des caractéristiques simples.
- Les couches profondes combinent ces caractéristiques pour représenter des concepts complexes.
- Un réseau profond peut approximer des fonctions très complexes (théorème d'approximation universelle).
- En pratique, les réseaux profonds sont plus efficaces et plus compacts que les réseaux peu profonds.

Partie 3: Fonctionnement interne

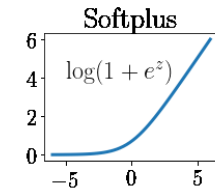
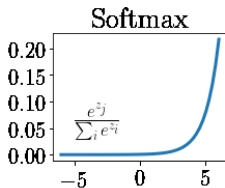
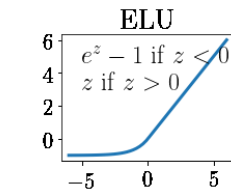
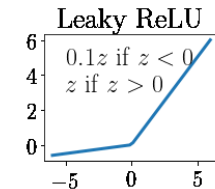
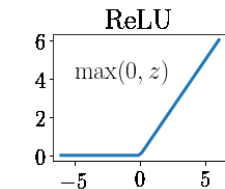
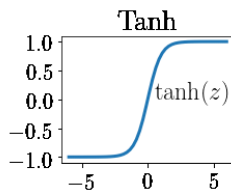
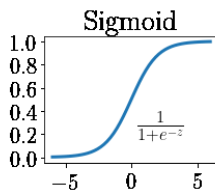
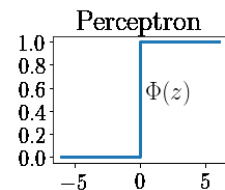
Fonctions d'activation

- Une fonction d'activation introduit de la non-linéarité dans le réseau de neurones.
- Sans fonction d'activation, un réseau multicouche serait équivalent à un modèle linéaire.
- Elle est appliquée à la sortie de chaque neurone :

$$a = \varphi(z) = \varphi(Wx + b)$$

- Les fonctions d'activation les plus courantes sont : Sigmoid, Tanh, ReLU, Softmax.
- Le choix de la fonction d'activation influence fortement les performances et la convergence du réseau.

Fonctions d'activation



Propagation avant (Forward Propagation) dans un MLP

- La propagation avant consiste à calculer la sortie du réseau à partir des données d'entrée.
- L'information traverse le réseau couche par couche, de l'entrée vers la sortie.
- À chaque couche l , deux étapes sont effectuées :

- **Combinaison linéaire :**

$$z^{(l)} = W^{(l)} a^{(l-1)} + b^{(l)}$$

- **Application de la fonction d'activation :**

$$a^{(l)} = \varphi^{(l)} \left(z^{(l)} \right)$$

- Les activations de départ sont les entrées du réseau :

$$a^{(0)} = x$$

- La sortie finale du réseau correspond à :

$$\hat{y} = a^{(L)}$$

- Cette étape permet d'obtenir la prédiction du modèle

Fonction de coût (Loss Function)

- La fonction de coût mesure l'erreur entre la prédiction du réseau et la vraie valeur.
- Elle permet de quantifier la qualité du modèle et de guider l'apprentissage.
- Pour une prédiction $\hat{y}$ et une cible y , la perte est donnée par :

$$\mathcal{L}(y, \hat{y})$$

- En pratique, on minimise la perte moyenne sur l'ensemble des données :

$$J(\theta) = \frac{1}{N} \sum_{i=1}^N \mathcal{L}(y_i, \hat{y}_i)$$

- L'objectif de l'apprentissage est de trouver les paramètres θ qui minimisent :

$$\theta^* = \arg \min_{\theta} J(\theta)$$

Optimisation en Deep Learning

- En Deep Learning, l'apprentissage consiste à optimiser des réseaux contenant des millions de paramètres.
- La fonction de coût est généralement hautement non linéaire et non convexe.
- Le paysage d'optimisation peut contenir de nombreux minima locaux, points selles et plateaux.
- Le calcul exact du minimum global est impossible en pratique.
- On utilise donc des méthodes itératives basées sur le gradient pour trouver une bonne solution approximative.
- L'efficacité de l'apprentissage dépend fortement du taux d'apprentissage, de l'initialisation et de l'architecture du réseau.

Le gradient en Deep Learning

- Le gradient indique comment la fonction de coût varie par rapport aux paramètres du réseau.
- Pour un réseau de neurones de paramètres θ , le gradient est :

$$\nabla_{\theta} J(\theta) = \left(\frac{\partial J}{\partial \theta_1}, \frac{\partial J}{\partial \theta_2}, \dots \right)$$

- Chaque composante du gradient mesure l'influence d'un paramètre sur l'erreur.
- Le gradient donne la direction de la plus forte augmentation de la fonction de coût.
- Pour minimiser la fonction de coût, on se déplace dans la direction opposée au gradient.
- En Deep Learning, le calcul efficace de ce gradient est rendu possible grâce à la rétropropagation.

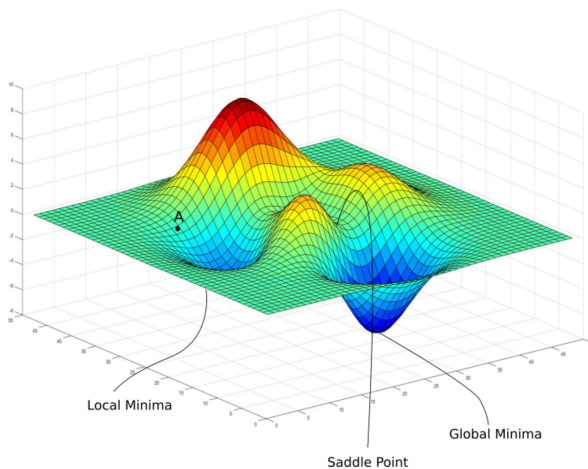
Descente de gradient

- La descente de gradient est un algorithme itératif utilisé pour minimiser la fonction de coût.
- À chaque itération, les paramètres du réseau sont mis à jour dans la direction opposée au gradient :

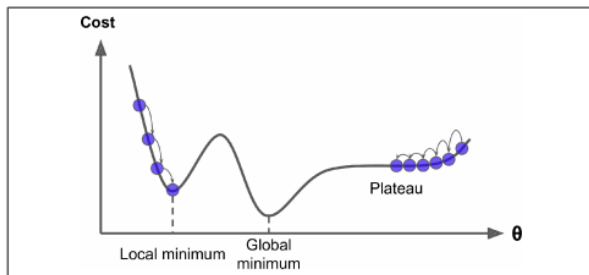
$$\theta \leftarrow \theta - \eta \nabla_{\theta} J(\theta)$$

- Où :
 - η est le taux d'apprentissage (learning rate)
 - $\nabla_{\theta} J(\theta)$ est le gradient de la fonction de coût
- Cette mise à jour est répétée jusqu'à convergence.
- Si η est trop grand, l'apprentissage diverge ; s'il est trop petit, l'apprentissage est très lent.

Descente de gradient



Descente de gradient



Descente de gradient stochastique (SGD)

- En pratique, on ne calcule pas le gradient sur toutes les données (trop coûteux).
- On utilise un sous-ensemble de données appelé mini-batch.
- La mise à jour des paramètres devient :

$$\theta \leftarrow \theta - \eta \nabla_{\theta} J_{\text{mini-batch}}(\theta)$$

- Avantages :
 - Beaucoup plus rapide
 - Permet de traiter de très grands ensembles de données
 - Introduit du bruit qui aide à sortir des minima locaux
- SGD est la méthode de base utilisée pour entraîner les réseaux profonds.

Règle de la chaîne et rétropropagation

- Un réseau de neurones est une composition de nombreuses fonctions :

$$x \rightarrow z^{(1)} \rightarrow a^{(1)} \rightarrow z^{(2)} \rightarrow \dots \rightarrow \hat{y} \rightarrow J$$

- La fonction de coût J dépend indirectement de tous les poids du réseau.
- Pour calculer l'influence d'un poids sur l'erreur, on utilise la règle de la chaîne :

$$\frac{\partial J}{\partial w} = \frac{\partial J}{\partial a} \cdot \frac{\partial a}{\partial z} \cdot \frac{\partial z}{\partial w}$$

- La rétropropagation consiste à propager l'erreur de la sortie vers l'entrée, couche par couche.
- Elle permet de calculer efficacement le gradient de tous les paramètres du réseau.

Exemple : neurone unique

Considérons un neurone avec :

$$x = 2, \quad w = 0.5, \quad b = 1$$

et une fonction d'activation identité : $\varphi(z) = z$.

1) Propagation avant :

$$z = wx + b = 0.5 \times 2 + 1 = 2$$

$$\hat{y} = z = 2$$

Supposons que la vraie valeur soit :

$$y = 3$$

Fonction de coût (erreur quadratique) :

$$J = (y - \hat{y})^2 = (3 - 2)^2 = 1$$

Exemple: neurone unique

2) Idée de la rétropropagation :

On veut savoir comment w influence l'erreur :

$$\frac{\partial J}{\partial w} = \frac{\partial J}{\partial \hat{y}} \cdot \frac{\partial \hat{y}}{\partial z} \cdot \frac{\partial z}{\partial w}$$

Ici :

$$\frac{\partial J}{\partial \hat{y}} = -2(y - \hat{y}) = -2$$

$$\frac{\partial \hat{y}}{\partial z} = 1$$

$$\frac{\partial z}{\partial w} = x = 2$$

Donc :

$$\frac{\partial J}{\partial w} = -4$$

Conclusion : le gradient indique comment modifier w pour réduire l'erreur.

Propagation avant et rétropropagation

- **Propagation avant (Forward) :**

$$x \rightarrow h^{(1)} \rightarrow h^{(2)} \rightarrow \hat{y} \rightarrow J$$

Elle permet de calculer la prédiction et l'erreur.

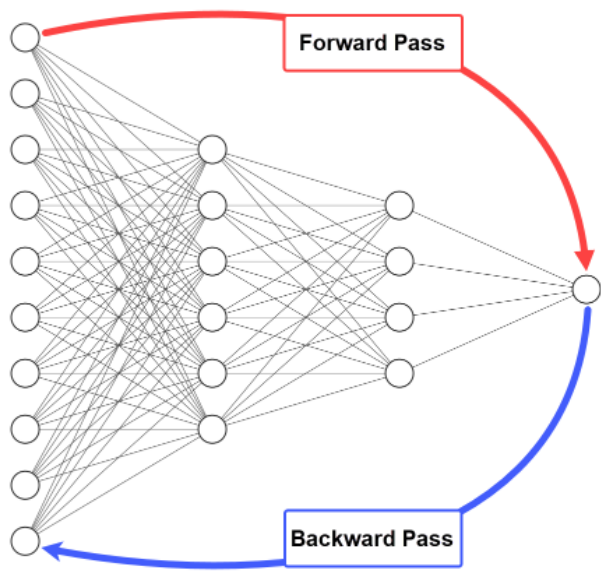
- **Rétropropagation (Backward) :**

$$J \rightarrow \hat{y} \rightarrow h^{(2)} \rightarrow h^{(1)} \rightarrow W$$

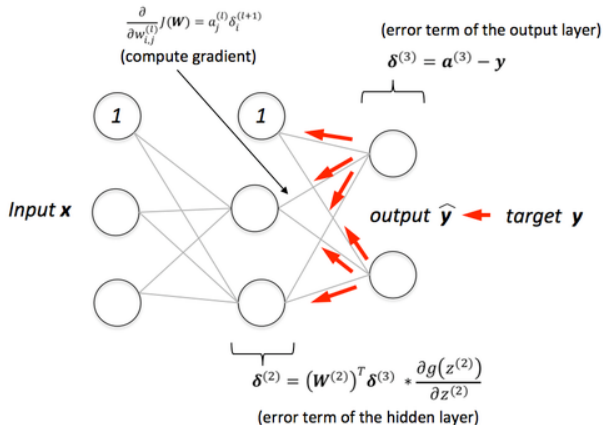
Elle permet de calculer les gradients des paramètres.

- Le forward calcule les valeurs, le backward calcule les dérivées.
- Les gradients sont ensuite utilisés pour mettre à jour les poids par descente de gradient.

Propagation avant et rétropropagation



Propagation avant et rétropropagation



Problèmes : Vanishing et Exploding Gradients

- Lors de la rétropropagation, les gradients sont multipliés de couche en couche (règle de la chaîne).
- **Vanishing gradient (gradient qui disparaît) :**
 - Les gradients deviennent très petits en remontant dans le réseau.
 - Les premières couches apprennent très lentement ou plus du tout.
 - Problème fréquent avec les fonctions sigmoid et tanh.
- **Exploding gradient (gradient qui explose) :**
 - Les gradients deviennent très grands.
 - Les poids deviennent instables et l'apprentissage diverge.
- **Conséquences :** apprentissage lent, instable ou impossible.
- **Solutions courantes :**
 - Fonctions d'activation ReLU
 - Bonne initialisation des poids
 - Normalisation (BatchNorm)

- Pour entraîner un réseau, on ne calcule pas toujours le gradient sur toutes les données.
- **Batch gradient descent :**
 - utilise toutes les données pour chaque mise à jour
 - précis mais très lent et coûteux
- **Stochastic gradient descent (SGD) :**
 - utilise un seul exemple à la fois
 - rapide mais très bruité et instable
- **Mini-batch gradient descent :**
 - utilise un petit groupe d'exemples (ex: 32, 64, 128)
 - compromis entre vitesse et stabilité
 - méthode la plus utilisée en Deep Learning

Notions clés : Epoch, Batch et Iteration

- **Dataset** : ensemble complet des données d'entraînement.
- **Batch / Mini-batch** :
 - Sous-ensemble du dataset (ex: 32, 64, 128 exemples)
 - Utilisé pour calculer un gradient et mettre à jour les poids
- **Iteration (ou Step)** :
 - Une itération = un passage d'un mini-batch dans le réseau
 - Donc : 1 forward + 1 backward + 1 mise à jour des poids
- **Epoch** :
 - Une époque = un passage complet sur tout le dataset
 - Elle contient plusieurs itérations
- Si le dataset contient N exemples et le batch size est B :

$$\text{Nombre d'itérations par époque} = \frac{N}{B}$$

Partie 5: Entraînement et généralisation

Cycle d'entraînement d'un réseau de neurones

- L'entraînement consiste à ajuster les paramètres du réseau pour minimiser la fonction de coût.
- Il se fait de manière itérative sur les données.
- Pour chaque mini-batch, on répète le cycle suivant :
 - ➊ Propagation avant (Forward) : calcul de la prédiction $\hat{y}$
 - ➋ Calcul de la perte : $\mathcal{L}(y, \hat{y})$
 - ➌ Rétropropagation (Backward) : calcul des gradients
 - ➍ Mise à jour des poids : descente de gradient

Données → **Forward** → **Loss** → **Backward** → **Update** → **Répéter**

- Ce cycle est répété pendant plusieurs époques jusqu'à convergence.

- **Underfitting (sous-apprentissage) :**
 - Le modèle est trop simple pour représenter les données.
 - Mauvaises performances sur les données d'entraînement et de test.
- **Overfitting (surapprentissage) :**
 - Le modèle s'adapte trop aux données d'entraînement.
 - Très bonnes performances sur le train, mauvaises sur le test.
 - Le modèle mémorise au lieu de généraliser.
- Objectif : trouver un bon compromis entre biais et variance.

Régularisation : L2, Dropout, Data Augmentation

- La **régularisation** regroupe les techniques qui améliorent la généralisation du modèle.

- **Régularisation L2 (Weight decay) :**

- Ajoute une pénalité sur la norme des poids :

$$J = J_{\text{data}} + \lambda \|W\|^2$$

- **Dropout :**

- Éteint aléatoirement certains neurones pendant l'entraînement.
 - Empêche le réseau de trop dépendre de neurones particuliers.

- **Data Augmentation :**

- Crée artificiellement de nouvelles données (rotation, bruit, etc.).

- L'early stopping est une technique simple pour éviter le surapprentissage.
- On surveille la performance sur l'ensemble de validation.
- Lorsque la performance de validation cesse de s'améliorer, on arrête l'entraînement.
- Cela permet :
 - d'éviter le surapprentissage
 - de gagner du temps de calcul

Évaluation d'un modèle : Train / Validation / Test

- On sépare généralement les données en trois ensembles :
 - **Ensemble d'entraînement (Train)** : pour apprendre les paramètres du modèle
 - **Ensemble de validation (Validation)** : pour régler les hyperparamètres
 - **Ensemble de test (Test)** : pour évaluer la performance finale
- Important :
 - Les données de test ne doivent jamais être utilisées pendant l'entraînement.
- Cela permet d'estimer la capacité de généralisation du modèle.

- **Accuracy (taux de bonne classification) :**

$$\text{Accuracy} = \frac{\text{Nombre de prédictions correctes}}{\text{Nombre total d'exemples}}$$

- **Matrice de confusion :**

- Vrai Positif (TP), Faux Positif (FP)
- Vrai Négatif (TN), Faux Négatif (FN)

- **Précision :**

$$\text{Precision} = \frac{TP}{TP + FP}$$

- **Rappel (Recall) :**

$$\text{Recall} = \frac{TP}{TP + FN}$$

- **F1-score :**

$$F1 = 2 \cdot \frac{\text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}}$$

- Si les classes sont déséquilibrées, l'accuracy peut être trompeuse.
- Exemple :
 - 99% de la classe A, 1% de la classe B
 - Un modèle qui prédit toujours A a 99% d'accuracy mais est inutile.
- Dans ce cas, on préfère utiliser :
 - Precision, Recall, F1-score
 - Courbe ROC et AUC

- **Erreur absolue moyenne (MAE) :**

$$\text{MAE} = \frac{1}{N} \sum_{i=1}^N |y_i - \hat{y}_i|$$

- **Erreur quadratique moyenne (MSE) :**

$$\text{MSE} = \frac{1}{N} \sum_{i=1}^N (y_i - \hat{y}_i)^2$$

- **Racine de l'erreur quadratique moyenne (RMSE) :**

$$\text{RMSE} = \sqrt{\text{MSE}}$$

- **Coefficient de détermination (R^2)**

Matrice de confusion

- La matrice de confusion permet d'analyser en détail les performances d'un classificateur.
- Pour un problème de classification binaire, elle contient :

	Prédit Positif	Prédit Négatif
Vrai Positif	TP (True Positive)	FN (False Negative)
Vrai Négatif	FP (False Positive)	TN (True Negative)

- Signification :
 - **TP** : bien prédit positif
 - **TN** : bien prédit négatif
 - **FP** : faux positif (fausse alarme)
 - **FN** : faux négatif (raté)
- À partir de cette matrice, on calcule :
 - Accuracy, Precision, Recall, F1-score